

EVENTOS DEL RATÓN (Mouse Events)

Ocurren al interactuar con el ratón.

- click → clic simple
- dblclick → doble clic
- mousedown → botón del ratón presionado
- mouseup → botón liberado
- mousemove → mover el ratón
- mouseover → el ratón entra en el elemento
- mouseout → el ratón sale
- mouseenter → entra (sin burbujeo)
- mouseleave → sale (sin burbujeo)
- contextmenu → clic derecho

EVENTOS DE TECLADO (Keyboard Events)

- keydown → tecla presionada
- keyup → tecla liberada

EVENTOS DE FORMULARIO (Form Events)

- focus → el campo recibe foco
- blur → pierde foco
- input → cambia el valor
- change → cambio confirmado
- select → texto seleccionado
- submit → enviar formulario
- reset → resetear formulario

EVENTOS DEL DOCUMENTO

Ocurren sobre el objeto document.

- DOMContentLoaded → se dispara cuando el HTML está completamente cargado
- readystatechange → se ejecuta cuando cambia el estado de carga del documento
- visibilitychange → se lanza cuando la pestaña pasa a visible u oculta (por ejemplo, al cambiar de pestaña).

EVENTOS DE TRANSICIONES CSS

Se producen durante una transición (transition).

- transitionstart → la transición CSS empieza.
- transitionend → la transición termina.
- transitionrun → la transición se prepara para ejecutarse.
- transitioncancel → la transición se cancela antes de terminar.

EVENTOS DE ANIMACIONES CSS

- animationstart → la animación comienza.
- animationend → la animación termina.
- animationiteration → la animación repite una vuelta.
- animationcancel → la animación se cancela.

EVENTOS MULTIMEDIA (audio/video)

Carga

- loadstart → empieza a cargarse el multimedia.
- loadeddata → se han cargado los primeros datos reproducibles.
- loadedmetadata → se cargan los metadatos (duración, dimensiones...).
- canplay → ya se puede reproducir (puede que se corte).
- canplaythrough → se puede reproducir entero sin cortes.

Reproducción

- play → el usuario pulsa reproducir.
- playing → el multimedia está reproduciéndose realmente.
- pause → se pausa la reproducción.
- ended → el multimedia termina.
- timeupdate → cambia el tiempo de reproducción (muy frecuente).
- volumechange → cambia el volumen.
- seeking → el usuario mueve la barra de tiempo.
- seeked → termina el cambio de posición.

FORMAS DE CAPTURAR EVENTOS

Hay **3 formas principales**:

En el HTML

El evento se escribe directamente en la etiqueta.

```
<button onclick="saludar()">Haz clic</button>
```

```
function saludar() {  
    alert("Hola");  
}
```

Con propiedad del DOM

Se asigna el evento desde JavaScript.

```
const boton = document.getElementById("miBoton");
```

```
boton.onclick = function() {  
    alert("Hola");  
};
```

Con addEventListener()

```
const boton = document.getElementById("miBoton");
```

```
boton.addEventListener("click", function() {  
    alert("Hola");  
});
```

MANEJADORES DE EVENTOS- HANDLER Y VARIABLE THIS

Un **manejador de eventos** (event handler) es la función que se ejecuta cuando ocurre un evento.

JavaScript define una variable especial llamada **this** que se crea automáticamente y que se emplea en algunas técnicas avanzadas de programación

Dentro de un manejador de eventos normal, **this** es el elemento HTML que recibió el evento.

Sirve para **actuar sobre el elemento que ha disparado el evento** sin tener que buscarlo otra vez.

Si quieres usar **this** en eventos no uses la función flecha =>

AÑADIR ESCUCHA

La sintaxis para agregar un handler:

element.addEventListener(event, handler, [options])

event: nombre del evento, por ejemplo: "click".

handler: la función handler.

options: un objeto adicional, opcional, con las propiedades:

- **once:** si es true entonces el listener se elimina automáticamente después de activarlo.
- **capture:** la fase en la que se controla el evento. Por razones históricas, **options** también puede ser false/true, lo que es igual a {capture: false/true}.
- **passive:** si es true entonces el handler no llamará a preventDefault().

Para eliminar el handler, usa `removeEventListener`:

`element.removeEventListener(event, handler, [options])`

OBJECT EVENT

El **objeto Event** es el objeto que JavaScript crea automáticamente cuando ocurre un evento.

Contiene **toda la información sobre lo que ha pasado**.

Sirve para obtener datos del evento, por ejemplo:

- qué elemento lo originó
- qué tecla se pulsó
- dónde se hizo clic
- tipo de evento

Para usarlo, simplemente debes declarar un parámetro en tu función (por convención se llama `e`, `evt` o `event`)

Propiedades Universales

Estas funcionan sin importar si hiciste click, pulsaste una tecla o cargaste la página.

- **type**: El nombre del evento (ej. "click", "submit").
- **target**: El elemento exacto donde se originó el evento (el botón que pulsaste).
- **currentTarget**: El elemento que tiene el `addEventListener` (el contenedor que "escucha").
- **bubbles**: Un booleano (`true/false`) que indica si el evento sube por el árbol del DOM (burbujeo).
- **timeStamp**: Milisegundos que han pasado desde que se cargó la página hasta el evento.
- **isTrusted**: `true` si el evento fue generado por un usuario (click real) y `false` si fue por código (`element.click()`).
- **clientX / clientY**: Coordenadas del ratón respecto a la **ventana** (viewport) del navegador.

- **pageX / pageY**: Coordenadas respecto a toda la **página** (incluye el scroll).
- **screenX / screenY**: Coordenadas respecto a la **pantalla física** de tu monitor.
- **button**: Qué botón se pulsó (0: izquierdo, 1: rueda, 2: derecho).
- **altKey, ctrlKey, shiftKey**: Devuelven true si esa tecla estaba presionada mientras hacías click.
- **key**: El carácter de la tecla (ej. "a", "Enter", "ArrowUp"). Es lo que el usuario "ve".
- **code**: El código de la tecla física (ej. "KeyA", "Digit1"). No cambia aunque cambies el idioma del teclado.
- **repeat**: true si la tecla se mantiene pulsada y el evento se está repitiendo.
- **location**: Indica en qué parte del teclado está la tecla (ej. para diferenciar el "Shift" izquierdo del derecho).
- **data**: El carácter que se acaba de introducir en un input.
- **inputType**: Cómo se insertó el texto (ej. "insertText", "insertFromPaste" si fue pegado).
- **formData**: (En eventos submit modernos) Permite acceder a todos los datos del formulario fácilmente.